ISEL
DEETC
Departamento de Engenharia Electrónica e de Telecomunicações e de Computadores

Aula 5

Sumário

- Circuitos combinatórios
- Projeto de circuitos combinatórios
- VHDL – descrição estrutural

Circuitos digitais

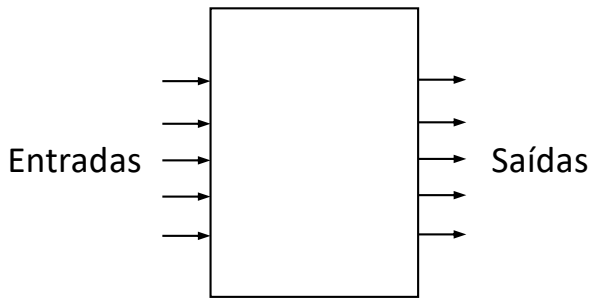
- Os circuitos digitais podem ser categorizados como:
  - Circuitos combinatórios
    - O valor da saída num determinado momento depende exclusivamente do valor das entradas nesse momento
  - Circuitos sequenciais
    - O valor da saída num determinado momento depende da sequência temporal do valor das entradas até esse momento

Circuitos combinatórios

- Um circuito combinatório é definido através da função lógica que representa cada uma das suas saídas
- É implementado utilizando portas lógicas que realizam as funções dos diversos operadores booleanos

Circuitos combinatórios

- Um circuito combinatório é definido através da função lógica que representa cada uma das suas saídas
- É implementado utilizando portas lógicas que realizam as funções dos diversos operadores booleanos



Hierarquia e reutilização de blocos funcionais

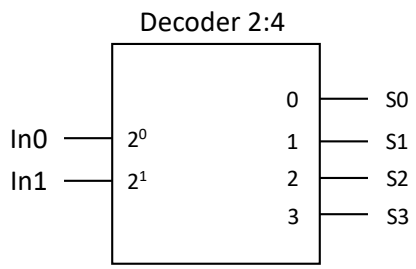
- No projeto de circuitos digitais complexos, a estratégia mais comum para desenhar a solução contempla o seguinte:
  - Dividir o problema em vários problemas mais pequenos (de resolução mais fácil)
  - Reutilizar blocos funcionais (módulos) previamente projetados:
    - Cada bloco funcional já está devidamente caracterizado e validado
    - Permite a redução do tempo necessário para o projeto
    - Reduz, potencialmente, os custos de produção
- O projeto modular utiliza vários módulos organizados hierarquicamente para construir o circuito lógico completo

Circuitos combinatórios típicos

- Exemplos de módulos frequentemente usados como base para circuitos com maior complexidade:
  - *Decoder* (descodificador)
  - *Encoder* (codificador)
  - *Multiplexer* (multiplexador)
  - *Demultiplexer* (desmultiplexador)

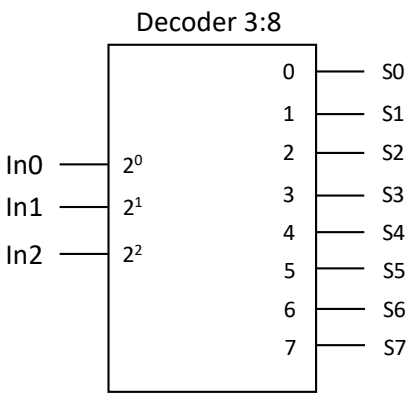
Decoder

- Converte *n* bits de entrada em *m* bits de saída ($n \leq m \leq 2^n$)
  - Tipicamente, converte *n* bits de entrada em 2^n bits de saída
- Cada código válido de entrada resulta numa única saída ativa
- Gera todos os mintermos dos bits de entrada
  - Exemplo:
    - Decoder de 2:4



Decoder 3-bit (3:8)

- Símbolo e tabela de verdade



IN2	IN1	IN0	S0	S1	S2	S3	S4	S5	S6	S7
0	0	0	1	0	0	0	0	0	0	0
0	0	1	0	1	0	0	0	0	0	0
0	1	0	0	0	1	0	0	0	0	0
0	1	1	0	0	0	1	0	0	0	0
1	0	0	0	0	0	0	1	0	0	0
1	0	1	0	0	0	0	0	1	0	0
1	1	0	0	0	0	0	0	0	1	0
1	1	1	0	0	0	0	0	0	0	1

Decoder 3-bit (3:8) Funções lógicas

- Funções das saídas

IN2	IN1	IN0	S0	S1	S2	S3	S4	S5	S6	S7
0	0	0	1	0	0	0	0	0	0	0
0	0	1	0	1	0	0	0	0	0	0
0	1	0	0	0	1	0	0	0	0	0
0	1	1	0	0	0	1	0	0	0	0
1	0	0	0	0	0	0	1	0	0	0
1	0	1	0	0	0	0	0	1	0	0
1	1	0	0	0	0	0	0	0	1	0
1	1	1	0	0	0	0	0	0	0	1

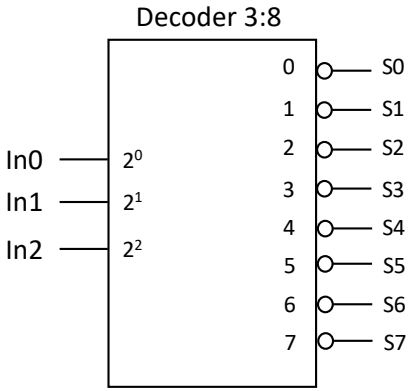
$$S0 = \overline{IN2} \cdot \overline{IN1} \cdot \overline{IN0}$$
$$S1 = \overline{IN2} \cdot \overline{IN1} \cdot IN0$$
$$S2 = \overline{IN2} \cdot IN1 \cdot \overline{IN0}$$
$$S3 = \overline{IN2} \cdot IN1 \cdot IN0$$
$$S4 = IN2 \cdot \overline{IN1} \cdot \overline{IN0}$$
$$S5 = IN2 \cdot \overline{IN1} \cdot IN0$$
$$S6 = IN2 \cdot IN1 \cdot \overline{IN0}$$
$$S7 = IN2 \cdot IN1 \cdot IN0$$

Circuitos digitais
níveis lógicos - *active_high* e *active_low*

- Os circuitos digitais utilizam grandezas físicas (e.g. tensão, corrente) para representar (codificar) os valores lógicos binários da álgebra de Boole
  - Cada tecnologia tem o seu próprio mapeamento
- Um dos mapeamentos tipicamente utilizados, consiste em fazer corresponder os valores lógicos binários a intervalos de tensão
  - Valor de tensão HIGH – [2; 5] V
  - Valor de tensão LOW – [0; 0,8] V
- Adicionalmente, para cada entrada/saída há que definir quando é que a mesma se considera ativa:
  - Valor de tensão HIGH – caso em que se denomina a entrada/saída como *active_high*
  - Valor de tensão LOW – caso em que se denomina a entrada/saída como *active_low*
- Para simplificar, consideraremos que:
  - o valor lógico numérico 1 está associado ao valor de tensão HIGH (1 <–> HIGH)
  - o valor lógico numérico 0 está associado ao valor de tensão LOW (0 <–> LOW)

Decoder 3-bit (3:8) com saídas *active_low*

- Símbolo e tabela de verdade



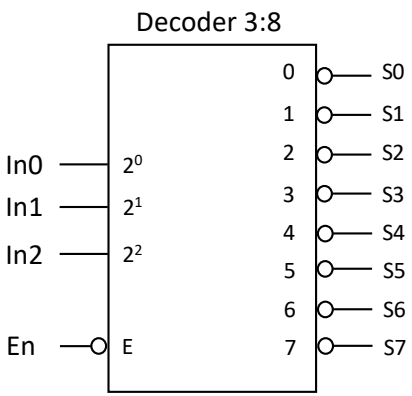
IN2	IN1	IN0	S0	S1	S2	S3	S4	S5	S6	S7
0	0	0	0	1	1	1	1	1	1	1
0	0	1	1	0	1	1	1	1	1	1
0	1	0	1	1	0	1	1	1	1	1
0	1	1	1	1	1	0	1	1	1	1
1	0	0	1	1	1	1	0	1	1	1
1	0	1	1	1	1	1	1	0	1	1
1	1	0	1	1	1	1	1	1	0	1
1	1	1	1	1	1	1	1	1	1	0

Entrada de *Enable*

- A entrada de *Enable* permite controlar quando o dispositivo executa a sua função normal
- Quando desactivada, o circuito apresenta na sua saída sempre o mesmo valor (independentemente do valor das outras entradas)
- O valor lógico das saídas, quando a entrada *Enable* está desactivada, é determinado pelo fabricante (normalmente corresponde ao valor lógico 0 caso a saída seja *active_high*, ou ao valor lógico 1 caso a saída seja *active_low)*
- Exemplo:
  - *Decoder* com *enable*

Decoder 3-bit (3:8) com saídas *active_low* e *enable* (*active_low*)

- Símbolo e tabela de verdade



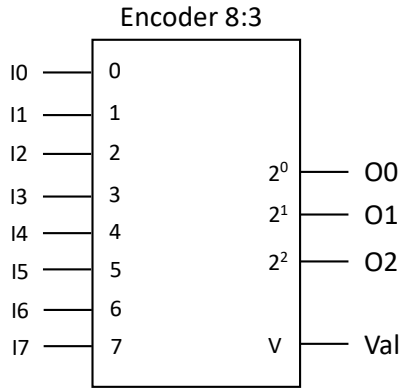
EN	IN2	IN1	IN0	S0	S1	S2	S3	S4	S5	S6	S7
1	-	-	-	1	1	1	1	1	1	1	1
0	0	0	0	0	1	1	1	1	1	1	1
0	0	0	1	1	0	1	1	1	1	1	1
0	0	1	0	1	1	0	1	1	1	1	1
0	0	1	1	1	1	1	0	1	1	1	1
0	1	0	0	1	1	1	1	0	1	1	1
0	1	0	1	1	1	1	1	1	0	1	1
0	1	1	0	1	1	1	1	1	1	0	1
0	1	1	1	1	1	1	1	1	1	1	0

Encoder

- Converte *m* bits de entrada em *n* bits de saída ($n \leq m \leq 2^n$)
  - Tipicamente, converte 2^n bits de entrada em *n* bits de saída
  - *n* representa o número de bits do palavra de código na saída
  - *m* representa o número de entradas distintas passíveis de serem codificadas
- Apenas considera uma entrada ativa a cada momento
- É a função inversa do *Decoder*

Encoder 8:3

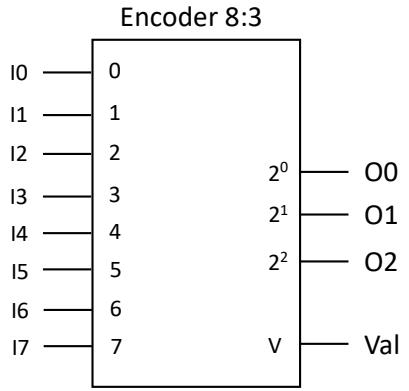
- Símbolo e tabela de verdade



I0	I1	I2	I3	I4	I5	I6	I7	O2	O1	O0	Val
0	0	0	0	0	0	0	0	-	-	-	0
1	0	0	0	0	0	0	0	0	0	0	1
0	1	0	0	0	0	0	0	0	0	1	1
0	0	1	0	0	0	0	0	0	1	0	1
0	0	0	1	0	0	0	0	0	1	1	1
0	0	0	0	1	0	0	0	1	0	0	1
0	0	0	0	0	1	0	0	1	0	1	1
0	0	0	0	0	0	1	0	1	1	0	1
0	0	0	0	0	0	0	1	1	1	1	1

Encoder 8:3

- Símbolo e tabela de verdade

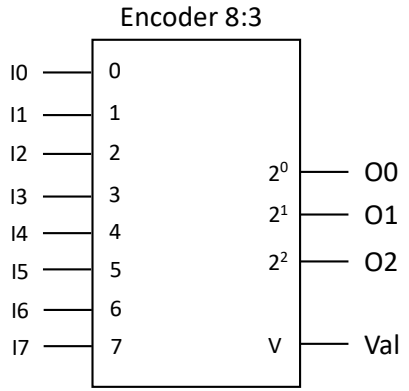


I0	I1	I2	I3	I4	I5	I6	I7	O2	O1	O0	Val
0	0	0	0	0	0	0	0	-	-	-	0
1	0	0	0	0	0	0	0	0	0	0	1
0	1	0	0	0	0	0	0	0	0	1	1
0	0	1	0	0	0	0	0	0	1	0	1
0	0	0	1	0	0	0	0	0	1	1	1
0	0	0	0	1	0	0	0	1	0	0	1
0	0	0	0	0	1	0	0	1	0	1	1
0	0	0	0	0	0	1	0	1	1	0	1
0	0	0	0	0	0	0	1	1	1	1	1

Tabela de verdade não está completa!!!
Faltam 247 combinações!!!

Encoder 8:3

- Símbolo e tabela de verdade



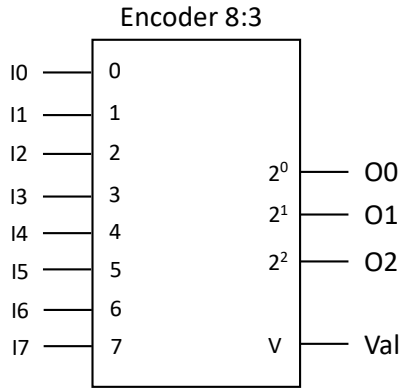
I0	I1	I2	I3	I4	I5	I6	I7	O2	O1	O0	Val
0	0	0	0	0	0	0	0	-	-	-	0
1	0	0	0	0	0	0	0	0	0	0	1
0	1	0	0	0	0	0	0	0	0	1	1
0	0	1	0	0	0	0	0	0	1	0	1
0	0	0	1	0	0	0	0	0	1	1	1
0	0	0	0	1	0	0	0	1	0	0	1
0	0	0	0	0	1	0	0	1	0	1	1
0	0	0	0	0	0	1	0	1	1	0	1
0	0	0	0	0	0	0	1	1	1	1	1
(restantes)								-	-	-	?

Priority encoder

- Converte *m* bits de entrada em *n* bits de saída ($n \leq m \leq 2^n$)
  - Tipicamente, converte 2^n bits de entrada em *n* bits de saída
  - *n* representa o número de bits do palavra de código na saída
  - *m* representa o número de entradas distintas passíveis de serem codificadas
- Permite várias entradas ativas simultaneamente, mas apenas o código da entrada com maior prioridade é disponibilizado na saída

Priority encoder 8:3

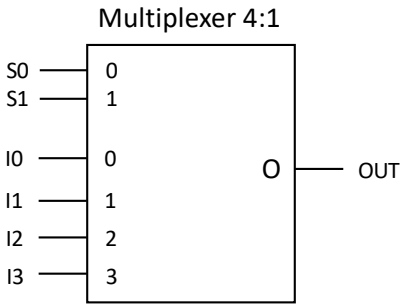
- Símbolo e tabela de verdade



I0	I1	I2	I3	I4	I5	I6	I7	O2	O1	O0	Val
0	0	0	0	0	0	0	0	-	-	-	0
1	-	-	-	-	-	-	-	0	0	0	1
0	1	-	-	-	-	-	-	0	0	1	1
0	0	1	-	-	-	-	-	0	1	0	1
0	0	0	1	-	-	-	-	0	1	1	1
0	0	0	0	1	-	-	-	1	0	0	1
0	0	0	0	0	1	-	-	1	0	1	1
0	0	0	0	0	0	1	-	1	1	0	1
0	0	0	0	0	0	0	1	1	1	1	1

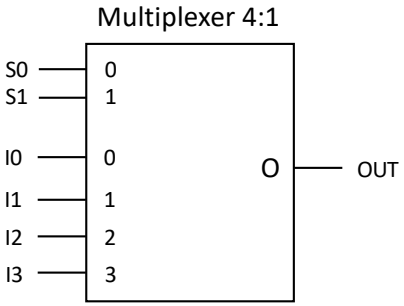
Multiplexer

- Seleciona uma de *n* entradas, em exclusão, para a saída
- A seleção é feita através de uma entrada de seleção com $\lceil \log_2 n \rceil$ bits que codifica a entrada a selecionar
- A implementação é feita à custa da descodificação da entrada de selecção
- Exemplo:

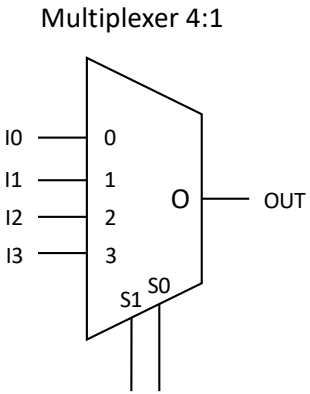


Multiplexer 4:1

- Símbolo e tabela de verdade



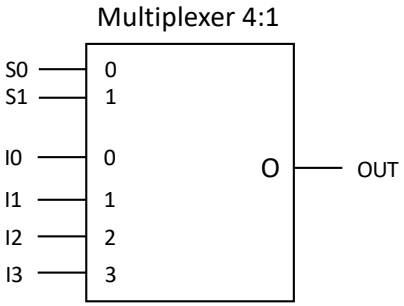
≡



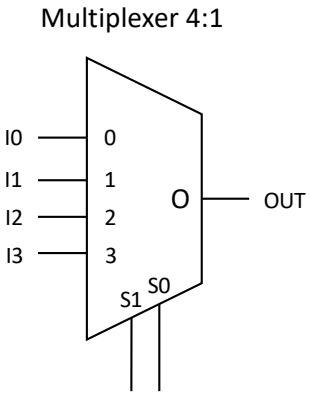
S1	S0	OUT
0	0	I0
0	1	I1
1	0	I2
1	1	I3

Multiplexer 4:1

- Símbolo e tabela de verdade



≡

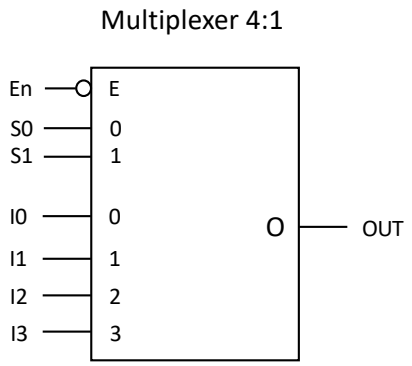


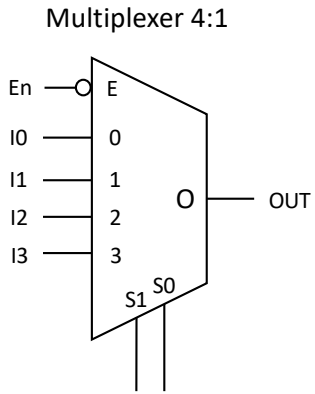
S1	S0	OUT
0	0	I0
0	1	I1
1	0	I2
1	1	I3

$$\mathrm{OUT} = \overline{\mathrm{S1}}\,\overline{\mathrm{S0}}\,\mathrm{I0} + \overline{\mathrm{S1}}\,\mathrm{S0}\,\mathrm{I1} + \mathrm{S1}\,\overline{\mathrm{S0}}\,\mathrm{I2} + \mathrm{S1}\,\mathrm{S0}\,\mathrm{I3}$$

Multiplexer 4:1 com *Enable active_low*

- Símbolo e tabela de verdade

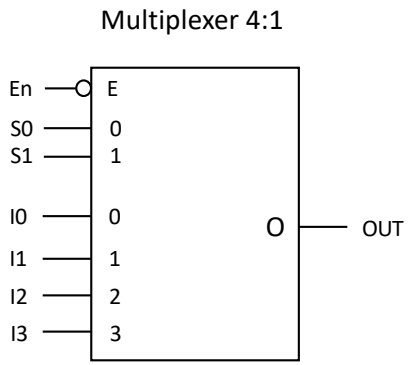


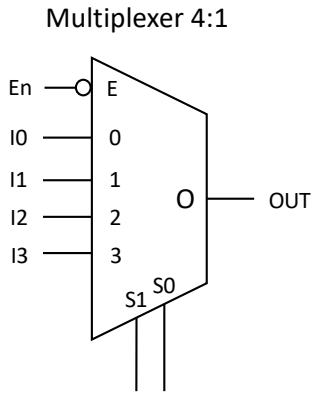


E	S1	S0	OUT
1	-	-	0
0	0	0	I0
0	0	1	I1
0	1	0	I2
0	1	1	I3

Multiplexer 4:1 com *Enable active_low*

- Símbolo e tabela de verdade



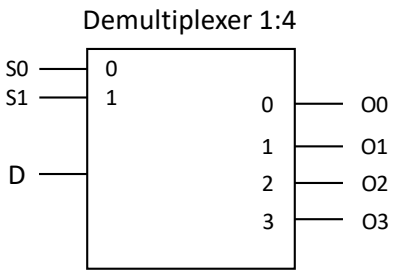


E	S1	S0	OUT
1	-	-	0
0	0	0	I0
0	0	1	I1
0	1	0	I2
0	1	1	I3

$$\mathrm{OUT} = \overline{\mathrm{EN}}\,(\overline{\mathrm{S1}}\,\overline{\mathrm{S0}}\,\mathrm{I0} + \overline{\mathrm{S1}}\,\mathrm{S0}\,\mathrm{I1} + \mathrm{S1}\,\overline{\mathrm{S0}}\,\mathrm{I2} + \mathrm{S1}\,\mathrm{S0}\,\mathrm{I3})$$

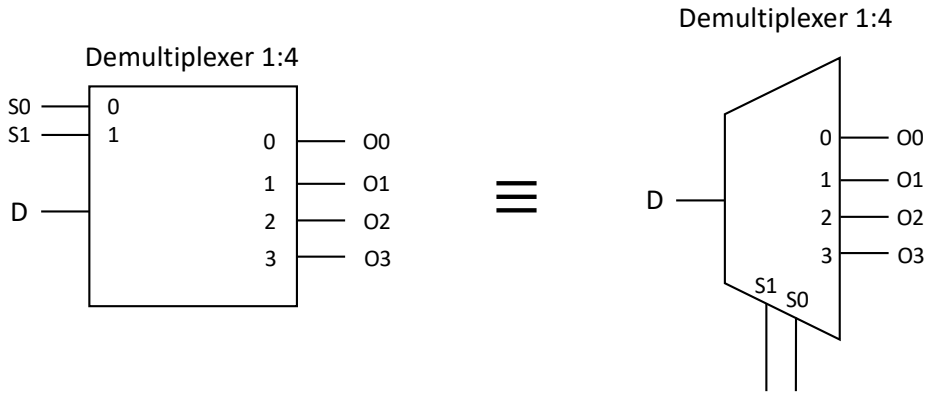
Demultiplexer

- Seleciona para qual das *n* saídas deve ser encaminhado o valor da entrada
- A selecção é feita através de uma entrada de selecção com $\lceil \log_2 n \rceil$ bits que codifica a saída a seleccionar
- A implementação é feita à custa da descodificação da entrada de selecção
- Exemplo:



Demultiplexer 1:4

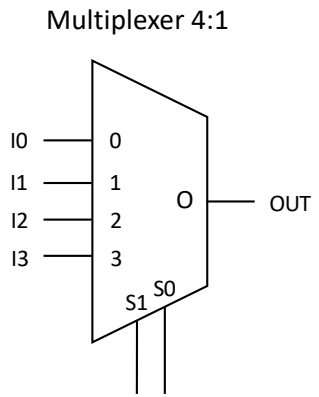
- Símbolo e tabela de verdade



S1	S0	O3	O2	O1	O0
0	0	0	0	0	D
0	1	0	0	D	0
1	0	0	D	0	0
1	1	D	0	0	0

Implementação de funções com *multiplexers*

- Um *multiplexer* pode ser utilizado para implementar funções lógicas
- Internamente, o *multiplexer* gera uma soma de produtos das variáveis de seleção
  - Exemplo: *multiplexer* 4:1

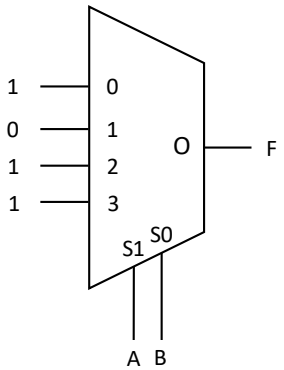


$$\mathrm{OUT} = \overline{\mathrm{S1}}\,\overline{\mathrm{S0}}\,\mathrm{I0} + \overline{\mathrm{S1}}\,\mathrm{S0}\,\mathrm{I1} + \mathrm{S1}\,\overline{\mathrm{S0}}\,\mathrm{I2} + \mathrm{S1}\,\mathrm{S0}\,\mathrm{I3}$$

Implementação de funções com *multiplexers*

- Um *multiplexer* 4:1 pode ser utilizado para implementar qualquer função lógica de 2 variáveis
- Consideremos a função descrita na tabela de verdade abaixo e a sua implementação com recurso a um *multiplexer* 4:1

A	B	F
0	0	1
0	1	0
1	0	1
1	1	1



$$F = \overline{S1}\,\overline{S0}\,I0 + \overline{S1}\,S0\,I1 + S1\,\overline{S0}\,I2 + S1\,S0\,I3$$

$$F = \overline{A}\,\overline{B}\,1 + \overline{A}\,B\,0 + A\,\overline{B}\,1 + A\,B\,1$$

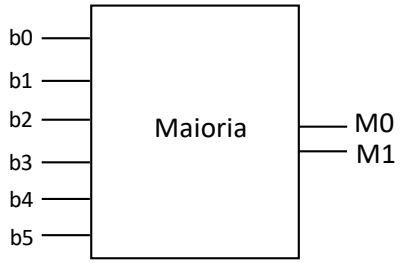
$$F = \overline{A}\,\overline{B} + A\,\overline{B} + A\,B$$

Projeto de circuitos combinatórios

- O projeto de um circuito combinatório com um pequeno número de entradas e saídas pode ser analisado por inteiro
- No entanto, para circuitos com maior número de entradas e saídas (maior complexidade), é preferível dividir o problema utilizando módulos funcionais mais simples (e devidamente caracterizados) que depois são interligados entre si para implementar a função desejada
- A subdivisão do circuito pode ter vários níveis de hierarquia

Projeto hierárquico (exemplo)

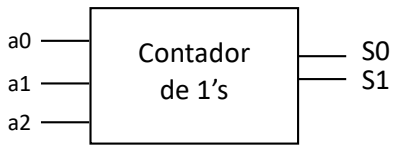
- Queremos implementar um circuito combinatório que indica se existem mais bits a 1 ou a 0 numa palavra de 6 bits. O resultado, codificado numa saída de 2 bits, é o seguinte:
  - 01 se #1's > #0's
  - 11 se #1's < #0's
  - 00 se #1's = #0's
  - E.g. – para a entrada "110011", a saída deverá ser "01"

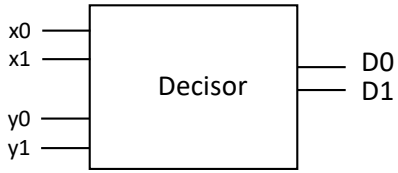


  - Duas funções com 6 entradas
    - Tabela de verdade com 6 entradas (64 possíveis combinações)

Projeto hierárquico (exemplo)

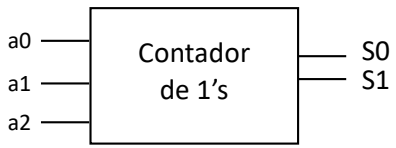
- Reduzir a complexidade (dividir o problema)
  - Utilizar blocos funcionais de menor complexidade



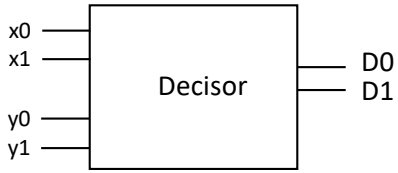


Projeto hierárquico (exemplo)

- Reduzir a complexidade (dividir o problema)
  - Utilizar blocos funcionais de menor complexidade



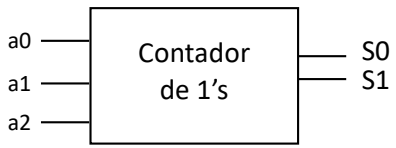
Duas funções de 3 entradas

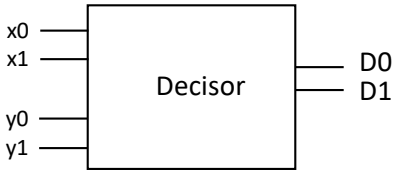


Duas funções de 4 entradas

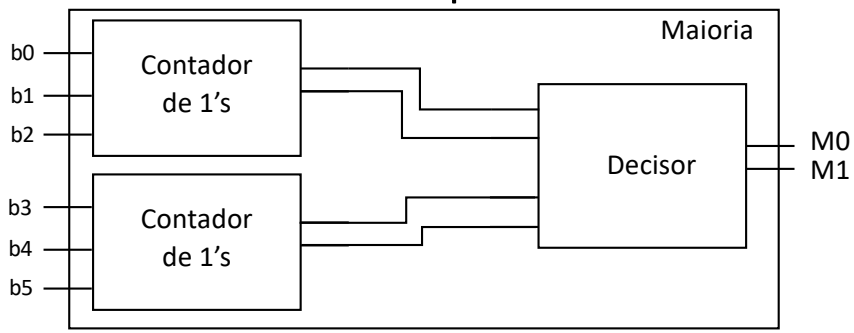
Projeto hierárquico (exemplo)

- Reduzir a complexidade (dividir o problema)
  - Utilizar blocos funcionais de menor complexidade





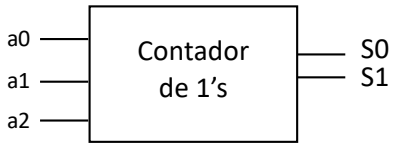
  - Desenhar circuito complexo à custa dos módulos mais simples



Projeto hierárquico (exemplo)

- Contador de 1's

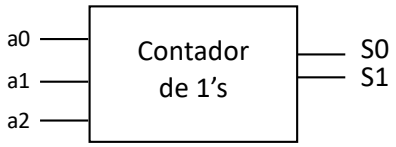
a2	a1	a0	S1	S0
0	0	0		
0	0	1		
0	1	0		
0	1	1		
1	0	0		
1	0	1		
1	1	0		
1	1	1		



Projeto hierárquico (exemplo)

- Contador de 1's

a2	a1	a0	S1	S0
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1



Projeto hierárquico (exemplo)

- Contador de 1's

a2	a1	a0	S1	S0
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

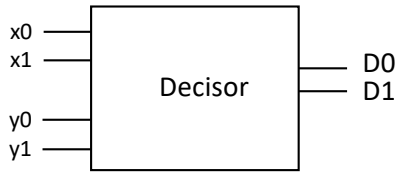
$$S0 = a2 \oplus a1 \oplus a0$$

$$S1 = a2\ a1 + a1\ a0 + a2\ a0$$

Projeto hierárquico (exemplo)

- Decisor

y1	y0	x1	x0	D1	D0
0	0	0	0		
0	0	0	1		
0	0	1	0		
0	0	1	1		
0	1	0	0		
0	1	0	1		
0	1	1	0		
0	1	1	1		
1	0	0	0		
1	0	0	1		
1	0	1	0		
1	0	1	1		
1	1	0	0		
1	1	0	1		
1	1	1	0		
1	1	1	1		

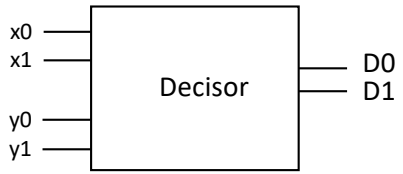


01 se #1's > #0's
11 se #1's < #0's
00 se #1's = #0's

Projeto hierárquico (exemplo)

- Decisor

y1	y0	x1	x0	D1	D0
0	0	0	0	1	1
0	0	0	1	1	1
0	0	1	0	1	1
0	0	1	1	0	0
0	1	0	0	1	1
0	1	0	1	1	1
0	1	1	0	0	0
0	1	1	1	0	1
1	0	0	0	1	1
1	0	0	1	0	0
1	0	1	0	0	1
1	0	1	1	0	1
1	1	0	0	0	0
1	1	0	1	0	1
1	1	1	0	0	1
1	1	1	1	0	1



Projeto hierárquico (exemplo)

- Decisor

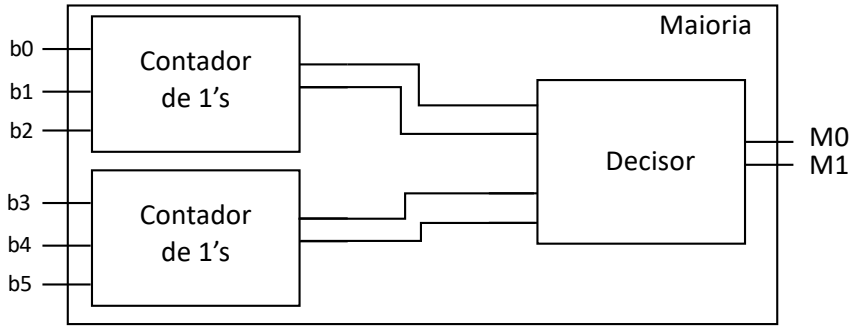
y1	y0	x1	x0	D1	D0
0	0	0	0	1	1
0	0	0	1	1	1
0	0	1	0	1	1
0	0	1	1	0	0
0	1	0	0	1	1
0	1	0	1	1	1
0	1	1	0	0	0
0	1	1	1	0	1
1	0	0	0	1	1
1	0	0	1	0	0
1	0	1	0	0	1
1	0	1	1	0	1
1	1	0	0	0	0
1	1	0	1	0	1
1	1	1	0	0	1
1	1	1	1	0	1

$$D0 = \overline{y1}\,\overline{x1} + y1x1 + y0x0 + \overline{y0}\,\overline{x0}$$

$$D1 = \overline{y1}\,\overline{x1} + \overline{y1}\,\overline{y0}\,\overline{x0} + \overline{y0}\,\overline{x0}\,\overline{x1}$$

Projeto hierárquico (exemplo)

- Uma vez tendo definido e implementado os módulos funcionais mais simples, é possível implementar o circuito mais complexo, fazendo as devidas interligações

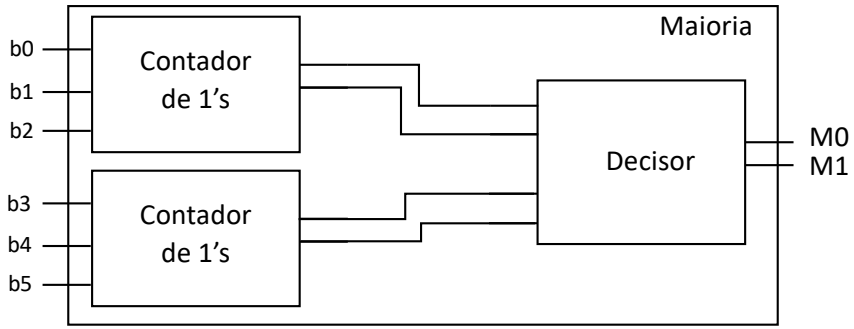


Projeto hierárquico (exemplo) VHDL – descrição estrutural

- Numa descrição do tipo *Behavioral* (comportamental) usam-se primitivas que descrevem o funcionamento da *entity* (entidade)
  - E.g. descrevem-se as funções lógicas de cada uma das saídas
- Numa descrição do tipo *Structural* (estrutural) usam-se vários componentes e as respetivas interligações para implementar a funcão desejada
  - É o tipo de descrição usada quando temos um bloco funcional que é construído à custa de módulos mais simples (projeto hierárquico)

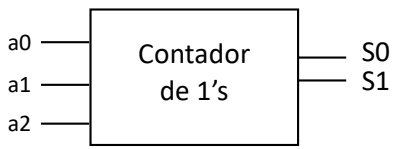
Projeto hierárquico (exemplo) VHDL – descrição estrutural

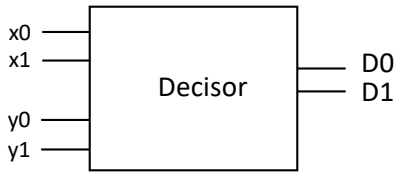
- Diagrama de blocos do circuito a implementar:



Projeto hierárquico (exemplo) VHDL – descrição estrutural

- Funções lógicas de cada um dos módulos funcionais mais simples:





$$S0 = a2 \oplus a1 \oplus a0$$

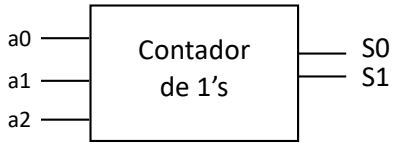
$$S1 = a2\,a1 + a1\,a0 + a2\,a0$$

$$D0 = \overline{y1}\,\overline{x1} + y1x1 + y0x0 + \overline{y0}\,\overline{x0}$$

$$D1 = \overline{y1}\,\overline{x1} + \overline{y1}\,\overline{y0}\,\overline{x0} + \overline{y0}\,\overline{x0}\,\overline{x1}$$

Projeto hierárquico (exemplo) VHDL – descrição estrutural

- Para cada um dos módulos funcionais, fazer a sua descrição em VHDL (comportamental) – uma entidade para cada módulo
  - Se já existir uma descrição prévia do módulo, reutilizá-la



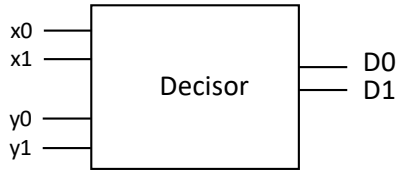
$$S0 = a2 \oplus a1 \oplus a0$$

$$S1 = a2\ a1 + a1\ a0 + a2\ a0$$

```
library ieee;
use ieee.std_logic_1164.all;

entity oneCounter is
   port (
      a : in std_logic_vector(2 downto 0);
      S : out std_logic_vector(1 downto 0)
   );
end oneCounter;


architecture behavioral of oneCounter is
begin

   S(0) <= a(2) xor a(1) xor a(0);
   S(1) <= (a(2) and a(1)) or (a(1) and a(0)) or (a(2) and a(0));

end behavioral;
```

Projeto hierárquico (exemplo) VHDL – descrição estrutural

- Para cada um dos módulos funcionais, fazer a sua descrição em VHDL (comportamental) – uma entidade para cada módulo
  - Se já existir uma descrição prévia do módulo, reutilizá-la



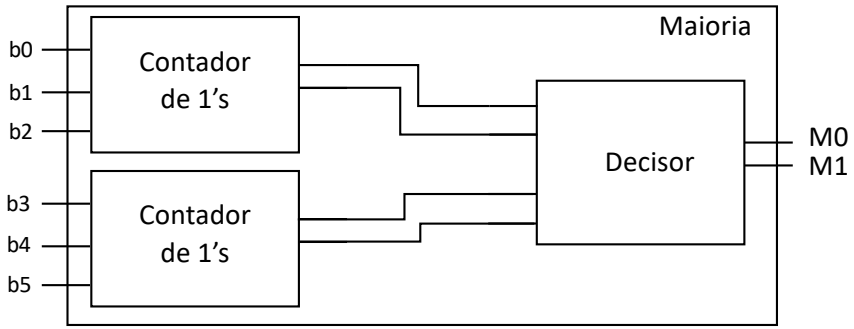
$$D0 = \overline{y1}\,\overline{x1} + y1x1 + y0x0 + \overline{y0}\,\overline{x0}$$

$$D1 = \overline{y1}\,\overline{x1} + \overline{y1}\,\overline{y0}\,\overline{x0} + \overline{y0}\,\overline{x0}\,\overline{x1}$$

```vhdl
library ieee;
use ieee.std_logic_1164.all;

entity decisionLogic is
   port (
      x : in std_logic_vector(1 downto 0);
      y : in std_logic_vector(1 downto 0);
      D : out std_logic_vector(1 downto 0)
   );
end decisionLogic;

architecture behavioral of decisionLogic is

   signal ny1nx1, ny0nx0 : std_logic;

begin

   ny1nx1 <= not y(1) and not x(1);
   ny0nx0 <= not y(0) and not x(0);

   D(0) <= ny1nx1 or (y(1) and x(1)) or (y(0) and x(0)) or ny0nx0;
   D(1) <= ny1nx1 or (not y(1) and ny0nx0) or (ny0nx0 and not x(1));

end behavioral;
```

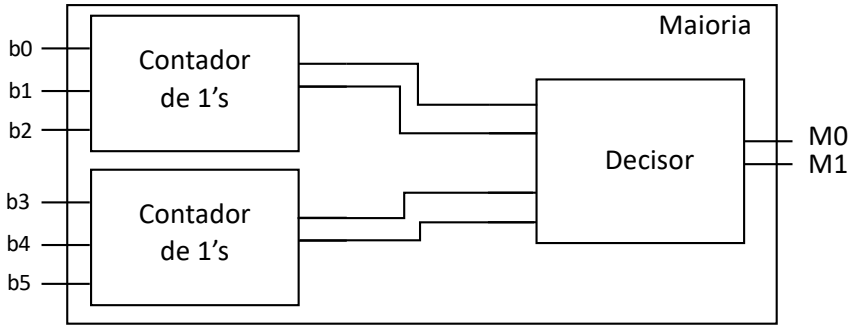
Projeto hierárquico (exemplo) VHDL – descrição estrutural

- O circuito “Maioria” é construído à custa de ligações entre vários módulos:
  - 2x Contador de 1’s
  - 1x Decisor



Projeto hierárquico (exemplo) VHDL – descrição estrutural

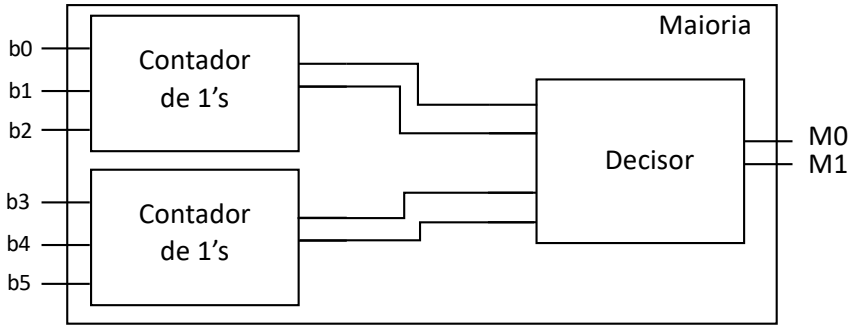
- Definir as entradas e saídas da entidade maioria (*majority*)




```vhdl
library ieee;
use ieee.std_logic_1164.all;

entity majority is
   port (
      b : in std_logic_vector(5 downto 0);
      M : out std_logic_vector(1 downto 0)
   );
end majority;


architecture structural of majority is


begin


end structural;
```

Projeto hierárquico (exemplo) VHDL – descrição estrutural

- Declarar os componentes que vão ser usados na descrição da arquitetura




```vhdl
library ieee;
use ieee.std_logic_1164.all;

entity majority is
   port (
      b : in std_logic_vector(5 downto 0);
      M : out std_logic_vector(1 downto 0)
   );
end majority;


architecture structural of majority is

   component oneCounter
      port (
         a : in std_logic_vector(2 downto 0);
         S : out std_logic_vector(1 downto 0)
      );
   end component;

   component decisionLogic
      port (
         x : in std_logic_vector(1 downto 0);
         y : in std_logic_vector(1 downto 0);
         D : out std_logic_vector(1 downto 0)
      );
   end component;

begin


end structural;
```

Projeto hierárquico (exemplo) VHDL – descrição estrutural

- A primitiva “component” é utilizada para declarar os componentes que serão utilizados na descrição de uma arquitetura (descrição estrutural)

```vhdl
library ieee;
use ieee.std_logic_1164.all;

entity oneCounter is
   port (
      a : in std_logic_vector(2 downto 0);
      S : out std_logic_vector(1 downto 0)
   );
end oneCounter;


architecture behavioral of oneCounter is
begin

   S(0) <= a(2) xor a(1) xor a(0);
   S(1) <= (a(2) and a(1)) or (a(1) and a(0)) or (a(2) and a(0));

end behavioral;
```

O nome e os portos têm de ser iguais

```vhdl
library ieee;
use ieee.std_logic_1164.all;

entity majority is
   port (
      b : in std_logic_vector(5 downto 0);
      M : out std_logic_vector(1 downto 0)
   );
end majority;


architecture structural of majority is

   component oneCounter
      port (
         a : in std_logic_vector(2 downto 0);
         S : out std_logic_vector(1 downto 0)
      );
   end component;

   component decisionLogic
      port (
         x : in std_logic_vector(1 downto 0);
         y : in std_logic_vector(1 downto 0);
         D : out std_logic_vector(1 downto 0)
      );
   end component;

begin


end structural;
```

Projeto hierárquico (exemplo) VHDL – descrição estrutural

- A primitiva “component” é utilizada para declarar os componentes que serão utilizados na descrição de uma arquitetura (descrição estrutural)

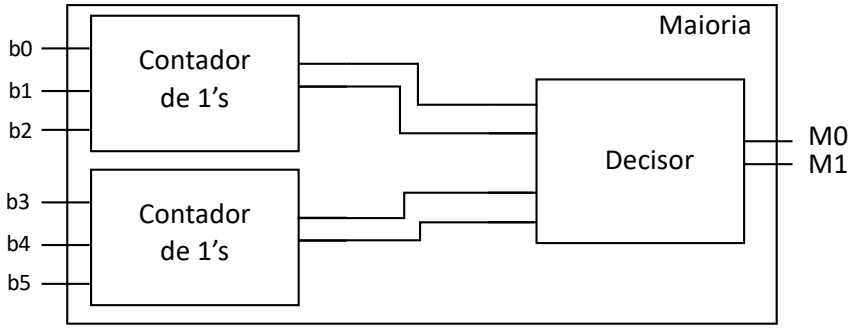
```vhdl
library ieee;
use ieee.std_logic_1164.all;

entity decisionLogic is
   port (
      x : in std_logic_vector(1 downto 0);
      y : in std_logic_vector(1 downto 0);
      D : out std_logic_vector(1 downto 0)
   );
end decisionLogic;

architecture behavioral of decisionLogic is

   signal ny1nx1, ny0nx0 : std_logic;

begin

   ny1nx1 <= not y(1) and not x(1);
   ny0nx0 <= not y(0) and not x(0);

   D(0) <= ny1nx1 or (y(1) and x(1)) or (y(0) and x(0)) or ny0nx0;
   D(1) <= ny1nx1 or (not y(1) and ny0nx0) or (ny0nx0 and not x(1));

end behavioral;
```

O nome e os portos têm de ser iguais

```vhdl
library ieee;
use ieee.std_logic_1164.all;

entity majority is
   port (
      b : in std_logic_vector(5 downto 0);
      M : out std_logic_vector(1 downto 0)
   );
end majority;


architecture structural of majority is

   component oneCounter
      port (
         a : in std_logic_vector(2 downto 0);
         S : out std_logic_vector(1 downto 0)
      );
   end component;

   component decisionLogic
      port (
         x : in std_logic_vector(1 downto 0);
         y : in std_logic_vector(1 downto 0);
         D : out std_logic_vector(1 downto 0)
      );
   end component;

begin


end structural;
```

Projeto hierárquico (exemplo) VHDL – descrição estrutural

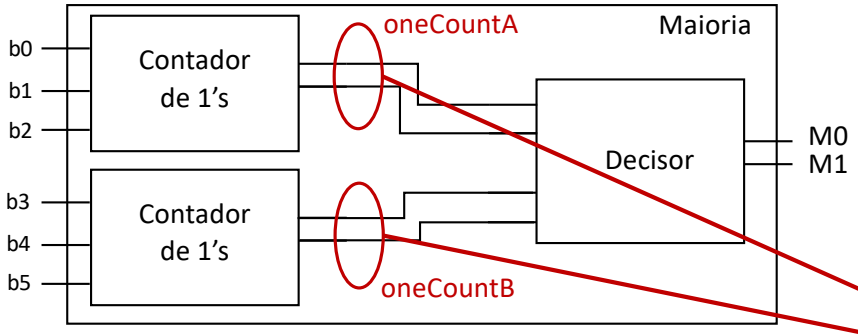
- Definir os sinais a serem utilizados




```vhdl
library ieee;
use ieee.std_logic_1164.all;

entity majority is
   port (
      b : in std_logic_vector(5 downto 0);
      M : out std_logic_vector(1 downto 0)
   );
end majority;


architecture structural of majority is

   component oneCounter
      port (
         a : in std_logic_vector(2 downto 0);
         S : out std_logic_vector(1 downto 0)
      );
   end component;

   component decisionLogic
      port (
         x : in std_logic_vector(1 downto 0);
         y : in std_logic_vector(1 downto 0);
         D : out std_logic_vector(1 downto 0)
      );
   end component;

   signal oneCountA, oneCountB : std_logic_vector (1 downto 0);

begin


end structural;
```

Projeto hierárquico (exemplo) VHDL – descrição estrutural

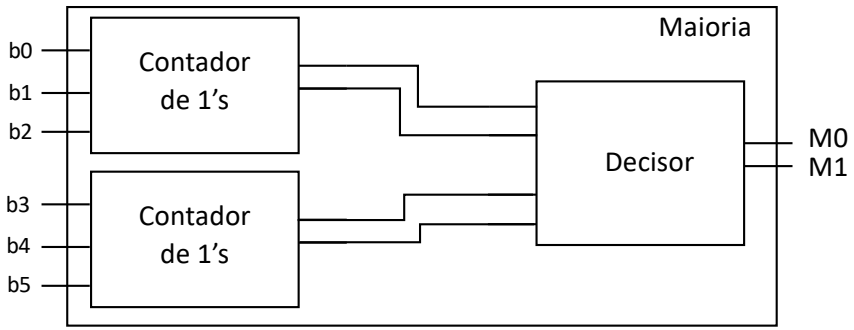
- Definir os sinais a serem utilizados




```vhdl
library ieee;
use ieee.std_logic_1164.all;

entity majority is
   port (
      b : in std_logic_vector(5 downto 0);
      M : out std_logic_vector(1 downto 0)
   );
end majority;


architecture structural of majority is

   component oneCounter
      port (
         a : in std_logic_vector(2 downto 0);
         S : out std_logic_vector(1 downto 0)
      );
   end component;

   component decisionLogic
      port (
         x : in std_logic_vector(1 downto 0);
         y : in std_logic_vector(1 downto 0);
         D : out std_logic_vector(1 downto 0)
      );
   end component;

   signal oneCountA, oneCountB : std_logic_vector (1 downto 0);

begin


end structural;
```

Projeto hierárquico (exemplo) VHDL – descrição estrutural

- Instanciar os componentes que forem necessários




```
begin

   oneCountUnitA : oneCounter port map (
      a => ,
      S =>
   );

   oneCountUnitB : oneCounter port map (
      a => ,
      S =>
   );

   decisionUnit : decisionLogic port map (
      x => ,
      y => ,
      D =>
   );

end structural;
```

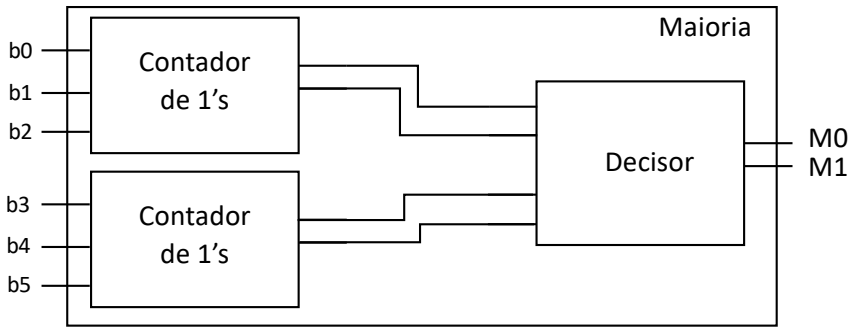
Projeto hierárquico (exemplo) VHDL – descrição estrutural

- Instanciar os componentes que forem necessários

Nome da instância
(tem de ser único)

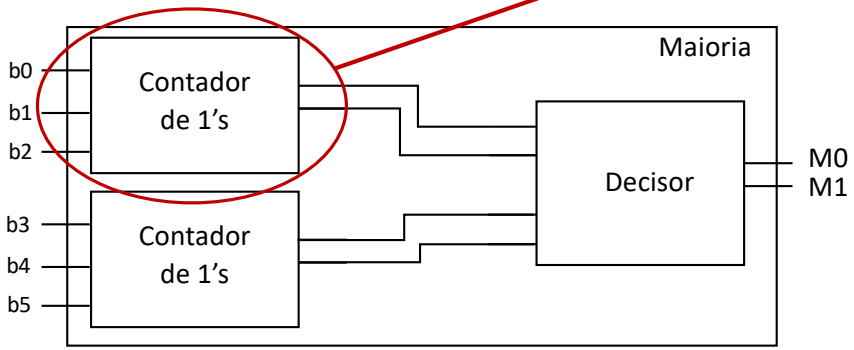
Componente a instanciar

```
begin

   oneCountUnitA : oneCounter port map (
      a => ,
      S =>
   );

   oneCountUnitB : oneCounter port map (
      a => ,
      S =>
   );

   decisionUnit : decisionLogic port map (
      x => ,
      y => ,
      D =>
   );

end structural;
```



Projeto hierárquico (exemplo) VHDL – descrição estrutural

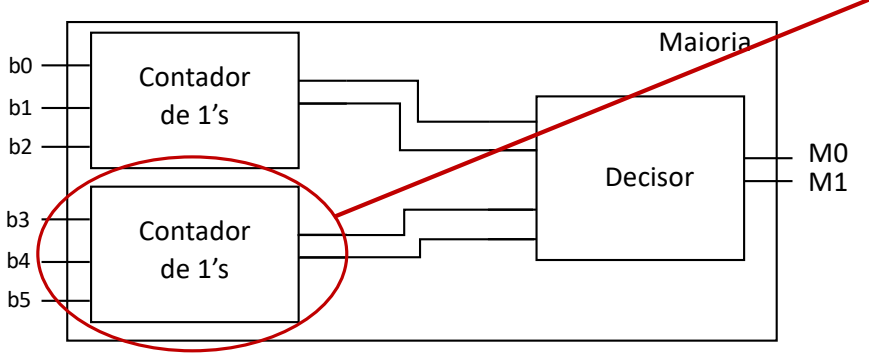
- Instanciar os componentes que forem necessários




```vhdl
begin

   oneCountUnitA : oneCounter port map (
      a => ,
      S =>
   );

   oneCountUnitB : oneCounter port map (
      a => ,
      S =>
   );

   decisionUnit : decisionLogic port map (
      x => ,
      y => ,
      D =>
   );

end structural;
```

Projeto hierárquico (exemplo) VHDL – descrição estrutural

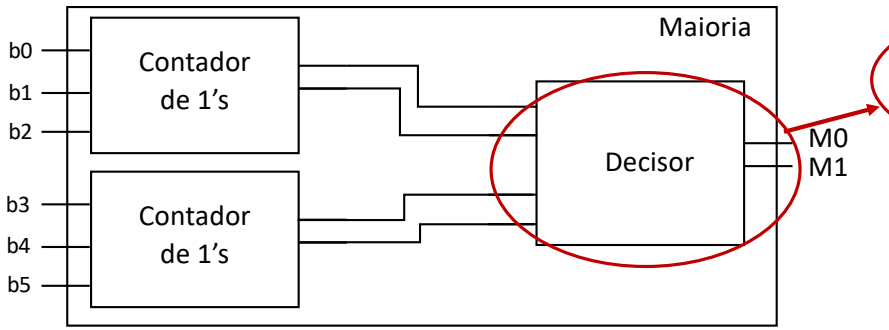
- Instanciar os componentes que forem necessários




```
begin

   oneCountUnitA : oneCounter port map (
      a => ,
      S =>
   );

   oneCountUnitB : oneCounter port map (
      a => ,
      S =>
   );

   decisionUnit : decisionLogic port map (
      x => ,
      y => ,
      D =>
   );

end structural;
```

Projeto hierárquico (exemplo) VHDL – descrição estrutural

- Instanciar os componentes que forem necessários




```
begin

   oneCountUnitA : oneCounter port map (
      a => ,
      S =>
   );

   oneCountUnitB : oneCounter port map (
      a => ,
      S =>
   );

   decisionUnit : decisionLogic port map (
      x => ,
      y => ,
      D =>
   );

end structural;
```

Projeto hierárquico (exemplo) VHDL – descrição estrutural

- Instanciar os componentes que forem necessários
- É necessário mapear (ligar) os portos dos componentes a entradas ou saídas ou sinais da entidade (*port map*)

```
begin

   oneCountUnitA : oneCounter port map (
      a => ???,
      S => ???
   );

   oneCountUnitB : oneCounter port map (
      a => ???,
      S => ???
   );

   decisionUnit : decisionLogic port map (
      x => ???,
      y => ???,
      D => ???
   );

end structural;
```

Projeto hierárquico (exemplo) VHDL – descrição estrutural

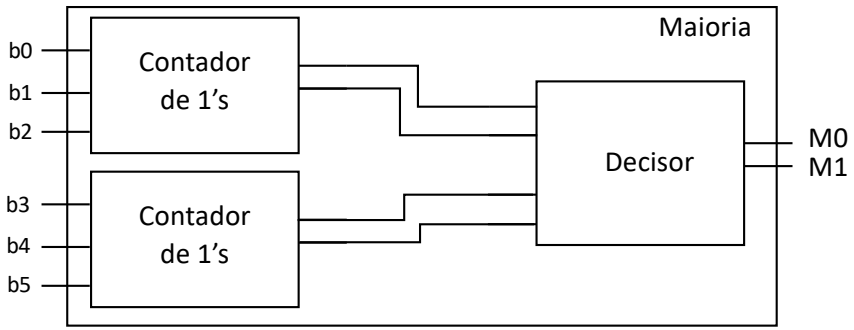
- Instanciar os componentes que forem necessários
- É necessário mapear (ligar) os portos dos componentes a entradas ou saídas ou sinais da entidade (*port map*)

```
component oneCounter
   port (
      a : in std_logic_vector(2 downto 0);
      S : out std_logic_vector(1 downto 0)
   );
end component;

component decisionLogic
   port (
      x : in std_logic_vector(1 downto 0);
      y : in std_logic_vector(1 downto 0);
      D : out std_logic_vector(1 downto 0)
   );
end component;
```

```
begin

   oneCountUnitA : oneCounter port map (
      a => ???,
      S => ???
   );

   oneCountUnitB : oneCounter port map (
      a => ???,
      S => ???
   );

   decisionUnit : decisionLogic port map (
      x => ???,
      y => ???,
      D => ???
   );

end structural;
```

Projeto hierárquico (exemplo) VHDL – descrição estrutural

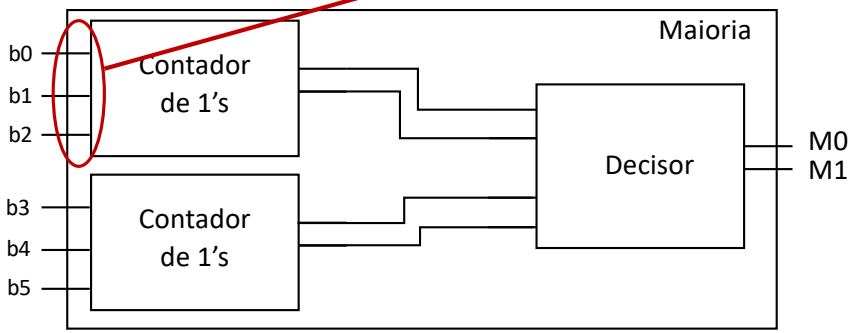
- Descrever a arquitetura à custa das ligações entre entradas e saídas da entidade e dos componentes internos




```vhdl
begin

   oneCountUnitA : oneCounter port map (
      a => b(2 downto 0),
      S => oneCountA
   );

   oneCountUnitB : oneCounter port map (
      a => b(5 downto 3),
      S => oneCountB
   );

   decisionUnit : decisionLogic port map (
      x => oneCountA,
      y => oneCountB,
      D => M
   );

end structural;
```

Projeto hierárquico (exemplo) VHDL – descrição estrutural

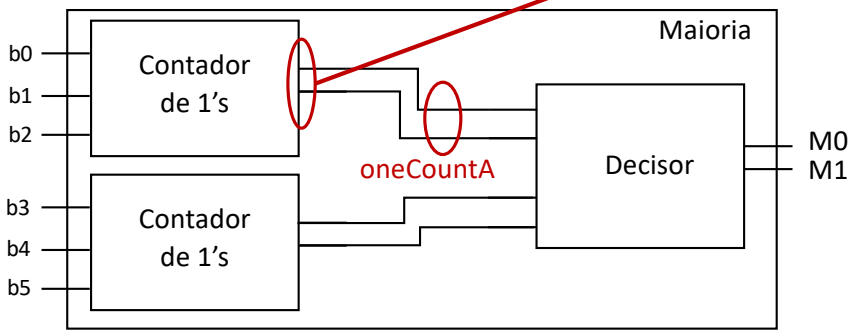
- Descrever a arquitetura à custa das ligações entre entradas e saídas da entidade e dos componentes internos




```vhdl
begin

   oneCountUnitA : oneCounter port map (
      a => b(2 downto 0),
      S => oneCountA
   );

   oneCountUnitB : oneCounter port map (
      a => b(5 downto 3),
      S => oneCountB
   );

   decisionUnit : decisionLogic port map (
      x => oneCountA,
      y => oneCountB,
      D => M
   );

end structural;
```

Projeto hierárquico (exemplo) VHDL – descrição estrutural

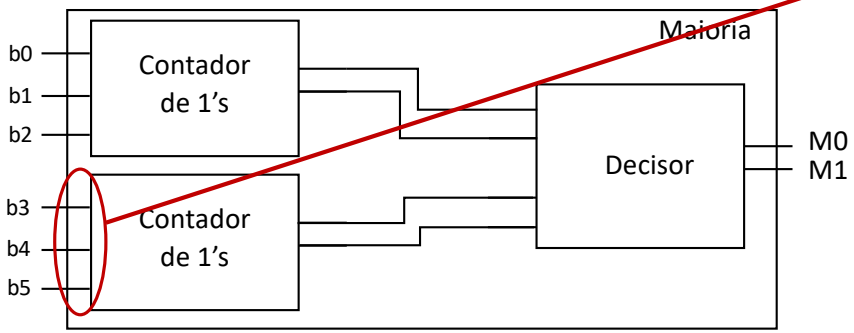
- Descrever a arquitetura à custa das ligações entre entradas e saídas da entidade e dos componentes internos




```vhdl
begin

   oneCountUnitA : oneCounter port map (
      a => b(2 downto 0),
      S => oneCountA
   );

   oneCountUnitB : oneCounter port map (
      a => b(5 downto 3),
      S => oneCountB
   );

   decisionUnit : decisionLogic port map (
      x => oneCountA,
      y => oneCountB,
      D => M
   );

end structural;
```

Projeto hierárquico (exemplo) VHDL – descrição estrutural

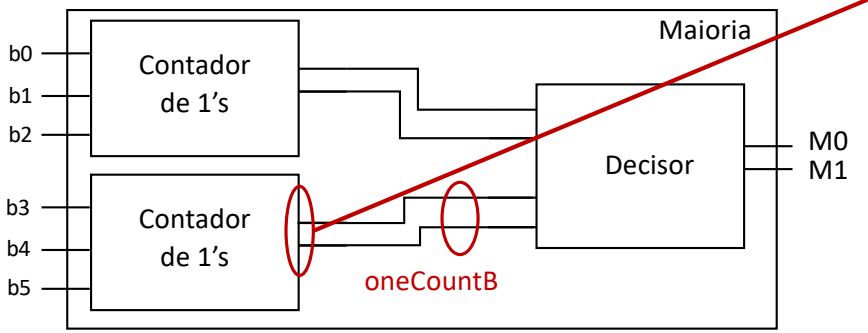
- Descrever a arquitetura à custa das ligações entre entradas e saídas da entidade e dos componentes internos




```
begin

   oneCountUnitA : oneCounter port map (
      a => b(2 downto 0),
      S => oneCountA
   );

   oneCountUnitB : oneCounter port map (
      a => b(5 downto 3),
      S => oneCountB
   );

   decisionUnit : decisionLogic port map (
      x => oneCountA,
      y => oneCountB,
      D => M
   );

end structural;
```

Projeto hierárquico (exemplo) VHDL – descrição estrutural

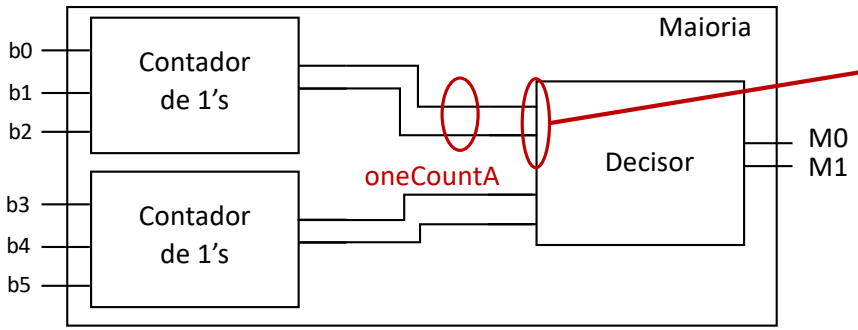
- Descrever a arquitetura à custa das ligações entre entradas e saídas da entidade e dos componentes internos




```
begin

   oneCountUnitA : oneCounter port map (
      a => b(2 downto 0),
      S => oneCountA
   );

   oneCountUnitB : oneCounter port map (
      a => b(5 downto 3),
      S => oneCountB
   );

   decisionUnit : decisionLogic port map (
      x => oneCountA,
      y => oneCountB,
      D => M
   );

end structural;
```

Projeto hierárquico (exemplo) VHDL – descrição estrutural

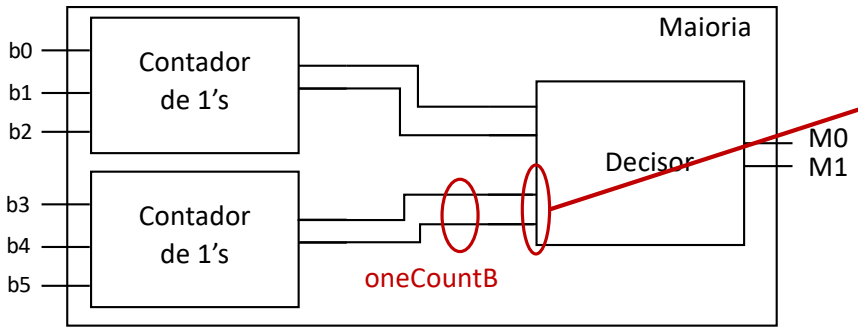
- Descrever a arquitetura à custa das ligações entre entradas e saídas da entidade e dos componentes internos




```vhdl
begin

   oneCountUnitA : oneCounter port map (
      a => b(2 downto 0),
      S => oneCountA
   );

   oneCountUnitB : oneCounter port map (
      a => b(5 downto 3),
      S => oneCountB
   );

   decisionUnit : decisionLogic port map (
      x => oneCountA,
      y => oneCountB,
      D => M
   );

end structural;
```

Projeto hierárquico (exemplo) VHDL – descrição estrutural

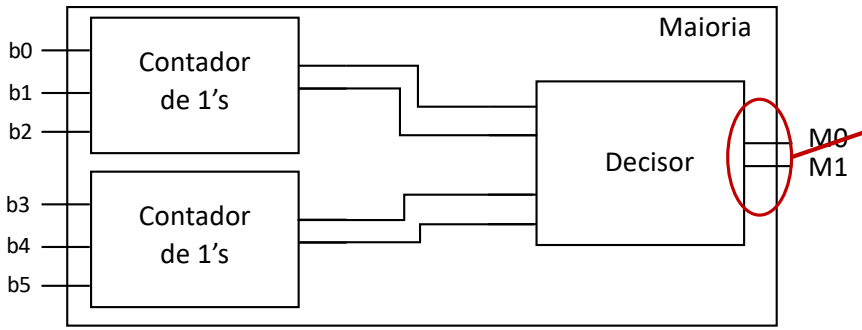
- Descrever a arquitetura à custa das ligações entre entradas e saídas da entidade e dos componentes internos




```vhdl
begin

   oneCountUnitA : oneCounter port map (
      a => b(2 downto 0),
      S => oneCountA
   );

   oneCountUnitB : oneCounter port map (
      a => b(5 downto 3),
      S => oneCountB
   );

   decisionUnit : decisionLogic port map (
      x => oneCountA,
      y => oneCountB,
      D => M
   );

end structural;
```

Projeto hierárquico (exemplo) VHDL – descrição estrutural

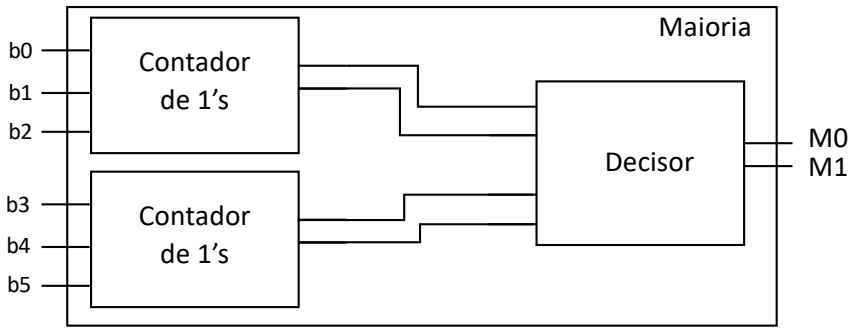
- Descrever a arquitetura à custa das ligações entre entradas e saídas da entidade e dos componentes internos




```vhdl
begin

   oneCountUnitA : oneCounter port map (
      a => b(2 downto 0),
      S => oneCountA
   );

   oneCountUnitB : oneCounter port map (
      a => b(5 downto 3),
      S => oneCountB
   );

   decisionUnit : decisionLogic port map (
      x => oneCountA,
      y => oneCountB,
      D => M
   );

end structural;
```

Projeto hierárquico (exemplo) VHDL – descrição estrutural

- Descrever a arquitetura à custa das ligações entre entradas e saídas da entidade e dos componentes internos




```vhdl
architecture structural of majority is

   component oneCounter
      port (
         a : in std_logic_vector(2 downto 0);
         S : out std_logic_vector(1 downto 0)
      );
   end component;

   component decisionLogic
      port (
         x : in std_logic_vector(1 downto 0);
         y : in std_logic_vector(1 downto 0);
         D : out std_logic_vector(1 downto 0)
      );
   end component;

   signal oneCountA, oneCountB : std_logic_vector (1 downto 0);

begin

   oneCountUnitA : oneCounter port map (
      a => b(2 downto 0),
      S => oneCountA
   );

   oneCountUnitB : oneCounter port map (
      a => b(5 downto 3),
      S => oneCountB
   );

   decisionUnit : decisionLogic port map (
      x => oneCountA,
      y => oneCountB,
      D => M
   );

end structural;
```

Temas abordados

Sumário

- Circuitos combinatórios
- Projeto de circuitos combinatórios
- VHDL – descrição estrutural

Próxima aula

- Laboratório B